

TCI 초기 컨셉 - 파트 2

0. 개발 목적/개발 배경

• 개발 배경

- 시장에는 '코드를 작성해주는 툴'과 '바꿔주는 툴'은 있지만, '정확하게 파악하고 가이드해주는 엔진 (Intelligence)'은 부재
- IDE, 정적 분석 툴, AI agent 툴 등에서 분석된 코드 정보가 연결되지 않고 휘발되기 때문에 이를 통합하여 지속 가능한 자산으로 관리할 지식 베이스 필요

• 개발 목적

- 기존 티맥스소프트가 애플리케이션의 실행을 책임지는 미들웨어(JEUS)로 시장을 장악했다면, 이제는 애플리케이션의 생산과 진화를 책임지는 **AI 코드 분석 인프라**로 포트폴리오를 확장
- 티맥스소프트 제품군의 **지속적인 기술 경쟁력 고양**

1. TCI가 지향하는 핵심 역할/가치는?

- **정체성:** TCI는 엔터프라이즈 애플리케이션에 흩어진 코드·문서·도메인 지식을 한데 모아, 시스템을 ‘한눈에 설명 가능한 구조’로 만들어주어 의사 결정을 지원하는 코드 인텔리전스
- **핵심 가치:**
 - 코드와 문서 등의 리소스에서 관계와 맥락을 해석해 통합된 Semantic Layer로 구축해 시스템을 ‘이해 가능한 대상’으로 변환
 - 신뢰성 있는 결정론적 문맥(Deterministic Context)을 제공하여 조직 차원의 기술 의사결정 지원
 - 외부 도구·AI 에이전트에 구조화 된 데이터를 제공

2. TCI의 주요 고객은 누구인가?

- **주요 사용자(사용자):** 개발/아키텍처/운영/품질 등 변경 영향, 의존성, 의사결정을 해야 하는 역할군
- **구매/도입 주체(기업, 조직):** 복잡한 코드베이스를 가진 엔터프라이즈 조직
 - **차세대 시스템 전환을 준비하는 기업**
 - 분석에만 수개월이 소요되는 대규모 코드베이스를 자동으로 이해하고, 전환 판단 근거를 확보하고 싶은 기업
 - **대형 SI/SM 프로젝트의 PM, PL**
 - 코드 수정이 전체 시스템에 미치는 영향을 통제하고 리스크를 사전에 관리하고 싶은 PM, PL
 - **TmaxSoft 내부 제품 및 플랫폼 조직**

- 제품의 개발 생산성과 제품 경쟁력을 높이고 싶은 내부 개발자

3. 어떤 문제를, 어떻게 해결하고자 하는지?

- 문제
 - 핵심 흐름/변경 영향도를 즉시 파악하기 어려움
 - 상호 의존성과 기술 의사결정에 필요한 근거가 부족함
 - 변경이 가져올 영향/리스크를 사전에 예측하기 어려움
- 해결 방식
 - 리소스(소스 코드, 문서, 비즈니스 도메인 등)를 분석하여 **통합된 데이터로 재구성**
 - **변경을 감지해 최신 상태 유지**
 - **사용자 질문 없이도 능동적이고 지속적으로 분석**
 - **자연어로 시스템 탐색 가능**
 - **답의 근거 제공** (추적 가능한 근거 기반)

4. 티맥스소프트 포트폴리오 관점에서 어떤 가치를 제공할 수 있는지?

- 티맥스소프트 내부 R&D의 개발 생산성과 제품 품질을 극대화하는 '분석 툴' 역할
 - **제품 지식의 자산화**: 개발자 개인에게 의존하던 암묵적인 도메인 지식을 TCI가 코드, 구조, 의존성 단위의 구조화된 데이터로 **추출, 저장**하여, 인력 변경에도 기술 부채가 누적되지 않도록 관리
 - **Dogfooding을 통한 제품 검증**: TCI를 티맥스소프트 제품 개발에 선제 적용하여 분석 정확도와 활용 가치를 내부에서 검증하고, “우리가 먼저 사용하고 검증한 제품”이라는 신뢰도 높은 레퍼런스 확보
 - **유지보수 및 장애 대응 효율화**: 고객센터에서 발생한 이슈 역추적 시 TCI의 영향도 분석 기능을 통해 **원인 분석 및 패치 소요 시간을 획기적으로 단축**
- 고객이 티맥스소프트 제품(ProFrame, ProObject 등)를 가장 잘 활용할 수 있도록 돕는 'AI 가이드' 역할
 - **프레임워크 사용 맥락 이해 지원**: 고객사는 티맥스소프트의 프레임워크(ProFrame 등) 위에서 자신들의 업무(여신, 수신 등)를 개발함. 하지만 프레임워크의 스펙이나 사용법을 완벽히 숙지하기 어려워 오용하거나 비효율적인 코드를 작성하는 경우가 많음. TCI가 기존 비즈니스 로직을 분석하여 **신규 프레임워크 환경으로 안전하게 안착할 수 있도록 네비게이션 제공**

5. 어떤 산업/업종에 최적화된 것인지?

- 금융

- 차세대 프로젝트, 계정계/정보계 분리 등 대규모 구조 변경 빈번
- 실패 비용이 매우 커, 정밀한 사전 분석이 필수

• 공공

- 장기간 누적된 레거시 코드
- 유지보수 주체 변경으로 코드 히스토리 단절
- 구조적 이해 없이 시스템 개편이 어려운 환경

기준	금융	공공	제조	이커머스/테크
레거시 비중	●●●	●●●	●●○	●○○
규제/감사 요구	●●●	●●●	●●○	●○○
장애 민감도	●●●	●●○	●●○	●●●
의사결정 속도	●○○	●○○	●●○	●●●
우리 경험	●●●	●●○	●○○	●○○